PYTHON FULL STACK DEVELOPMENT

PYTHON FULL STACK DEVELOPMENT COURSE

Course duration: 3 months

Course: Python,HTML5,CSS3,JavaScript and SQL

Trainer: Manikanta Machapalli

For more information contact us: 9886282230

“Believe in Yourself, Anything is Possible.”

A CONTENT GUIDE FOR BEGINNERS AND BEYOND

1.Introduction to python

- Installation and Environment settings
- Introduction to Shell

2. Variables, Keywords, Data Types & Identifiers

- Variable
- Keywords
- Data Types
- Identifiers

3. String, List, Tuple, Set, Dictionary and Slicing

- String Data Types
- List Data Types
- Tuple Data Types
- Set Data Types
- Dictionary Data Types
- Slicing Data Types

4. OPERATORS

- Arithmetic Operators
- Logical Operators
- Bitwise Operators
- Relational Operators
- Assignment Operators
- Membership Operators
- Identity Operators

5.Control Statements

- Decisional
- Looping
- Break, Continue & Pass
- 6.Input & Print Statements
- Input Statements
- Print Statements

6. FUNCTIONS OR METHODS

Types of Functions
Recursion
Arguments
Packing & Unpacking(Var args)

7. OOPS

Class, Objects
Inheritance

8.METHOD OVER RIDING

Access Specifiers
8. File Handling & JSON
Flat File Handling
Json
Pickle

9.EXCEPTION HANDLING

- Try
- Except & Final
- Custom Exceptions
- Raising Exceptions
- Assertions

10. MAP,FILTERS &LAMBDA EXPRESSIONS

11.ITERATORS,GENERATORS

12.COMPREHENSION

LANGUAGES:

Languages are used for communication between systems and humans

AP Native Language(Telugu)

Delhi Native Language(Hindi)

LOW LEVEL LANGUAGES

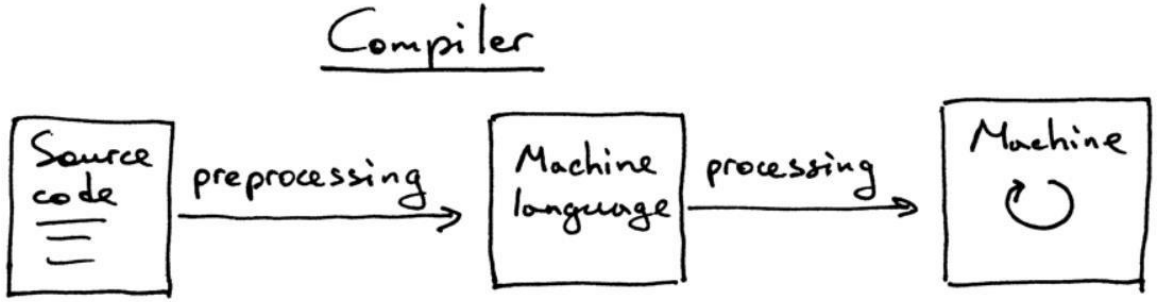
These Are Understandable By The Systems

HIGH LEVEL LANGUAGES

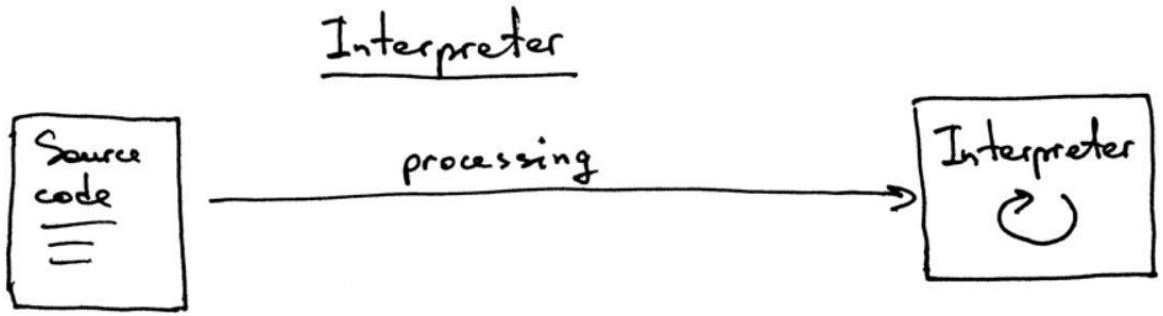
These are understandable by the humans

To convert HLL to LLL

Compilers



Interpretors



To convert HLL to LLL

High Level Language are divided into 2 types :1)Programming language
2)Scripting Language

1) Programming Language
In programming language we will use compiler

Compilers: Compiler scans the entire programme at a time
it converts .py source code to .pyc byte code
Compiler is faster than interpretor
Ex:c,c++,java

2) Scripting Language

In Scripting language we will use Interpretor

Interpretors: line by line compilation will be done by this Scripting Language.
it stores actual code
it is slower than compiler

Ex:Python,JavaScript

PYTHON

“Guido van Rossum” – 1980

- Python is a high level open source interpreted scripting language
- Python is a dynamic typed programming language
- Python is very easy to understand & learn
- Python has more built in libraries (1,37,000)
- Python is case sensitive language
- Python supports functional programming & object oriented programming language
- Python cross platform language

THESE ARE THE FIELDS IN WHICH WE HAVE USE PYTHON

Web Development
Artificial Intelligence
Data Science
Machine Learning
Internet of things
Automation ...etc

Editors of python

Pycharm
Visual Studio Code
Atom
Jupiter
Sublime Text
Ediplus

How to install python software

Search for python.org in your browser
Click on downloads --→ Click on download python 3.10.5 or latest version of the software
Once python extension file will be downloaded
Double click on the extension file
Please check the add python path check box
Now click on install now

IDLE [Integrated development and learning environment]

Idle stands for integrated development and learning environment

Idle is used to run the python programmes

Idle is equipped with all the default implementations of python language

WE CAN UTILIZE IN 2 WAYS

1. Python Shell
2. Python Module

1. Python Shell: It is an interactive console
   : We can execute only one statement at a time in python shell
2. Python Module: File with .py extension with the collection of multiple python statements

STEPS TO CREATE MODULE FROM SHELL

Open idle shell

Click on new and click on new file(cntrl+N)

Save the file without giving extension

COMMAND FOR EXECUTING PYTHON MODULE

Press F5 or Alt F5

STEPS FOR EXECUTING PYTHON MODULE

Python modulename.py

TOKENS

Tokens are the essential elements for writing the python program

TOKENS OF PYTHON

1.Keywords
2.Variables
3.Identifiers
4.DataTypes

1.KEYWORDS

- ✓ Keywords are reserved built in words which are defined for doing some specific task.
- ✓ True, None, False are in title case & remaining keywords will be in lower case
- ✓ We can't change the functionality of keywords
- ✓ In python we have 35 keywords
- ✓ To see the list of keywords follow the below commands

```
import keyword
keyword.kwlist
```

- ✓ ['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

If you want to find out how many keywords there are in Python, you can write this:

```
len(keyword.kwlist)
```

IMPORT KEYWORD

- **Import Keyword is used for performing the process of importing**
- **Importing is the process of accessing the content of another module into current module**

Ex:

Pavan.py

```
a=50
b=60
C=90
```

Anirudh.py

```
Import pavan
Print(pavan.a)
Print(pavan.b)
Print(pavan.a+pavan.b)
```

Variables
Variable is a name given to the memory allocation
The value stored in a variable might get varied

RIGHT TO LEFT

SYNTAX: Variablename=Value

NOTE:

- If we assign multiple variables with same integer value then they will point to same memory address
- If we assign new value for existing variable then it will point to new memory address
- Idle Shell
- a=90
- b=90
- id(a)
- 140705204884696
- id(b)
- 140705204884696
- a=80
- b=90
- a=80
- id(a)
- 140705204884376

ID FUNCTION
It is used for getting the address of the variable
SYNTAX
Id(Variablename)

GARBAGE COLLECTION
It is process of delete unused memory
DISADVANTAGES OF WHEN GC IS DONE BY DEVELOPERS
Deleting Memory
Deleting the memory before completion of its usage
Not deleting even after completion of its usage

PYTHON MEMORY MANAGEMENT
In python garbage collection will be done by PMM
Python memory management will delete the value when it has zero references

SYNTAX FOR DEFINING MULTIPLE VARIABLES
var1,var2,var3,----varN=value1,value2,value3,-------valueN
x,y,z=100,200,300

Swapping the variables

a=10
b=20
a,b=b,a

Swap the variables using third variable

c=a+b
a=c-a
B=c-b

Swap the variables without using third variable

a=a+b → a=10+20=30 → a=30
b=a-b → b=30-20=10 → b=10
a=a-b → a=30-10=20 → a=20

Identifiers

Identifier is a name which we can identify the variables, functions or classes

Rules to be followed while declaring the identifiers

First letter should not be use a number

We should not use keyword is an identifier

Other than underscore we cannot use any special characters in identifiers because each and every other special characters has some functionality in python

We can use combination of alphabets,numbers, and undercore

Data Types

Data Types are used for defining a type of data is stored in variable

Data Types

- Based on no of values stored in variable
  - Single Value Data Type
    - Number Dt
      - Integer
      - Float
      - Complex
    - Boolean Data Type
      - True
      - False
  - Multi Valued Collection Datatype
    - String
    - List
    - Tuple
    - Set
    - Dictionary
- Based on behaviour of value stored in variable
  - Immutable data type
    - all single valued Data Types
    - Tuple
    - String
  - Mutable Data Type
    - List
    - Set
    - Dictionary

Mutable data type(Variable length data type)
Immutable data type(fixed length data type)

Ordered Collection Data Type
Unordered Collection Data type

Indexing and slicing (possible or not possible)

String:’ ‘,” ”,’’’ ‘’’
List: [ ]
Tuple: ( )
Set: { } {10,20,30}
Dictionary: {keys:values}

Homo means same
Hetero means different
L=[10,20,10,20,10,20] [ ]
L=[10,20,30,40,50]

Del l[0]

Based on no of values stored in variable

1.Single Valued Data Type
2.Multi Valued Data Type

1.Single valued data type:
We can store only one single data in variable

1.Number Data Type
INTEGER: The Whole numbers without decimal points
FLOAT: The whole numbers with decimal points
COMPLEX: Complex numbers are the combination of real part and imaginary part
a+bj
BOOLEAN DATA TYPE:
These data types are used for defining true boolean type
True Keyword is used for defining true boolean data type
False keyword is used for defining false boolean data type

TYPE FUNCTION
It is used for returning the type of data stored in variable
Syntax:type(variable)

example: a=10
type(a)
<class 'int'>

Note Points:
Every thing in python is an object

```
a=10
type(a)
<class 'int'>
b=9.9
type(b)
<class 'float'>
c='manikanta'
type(c)
<class 'string'>
a=1+2j
type(a)
<class 'complex'>
d=True
type(d)
<class 'bool'>
d=False
<class 'bool'>
```

Indexing

in python we have both positive index positions and negative index positions

1.Positive Index Positions

Direction: Left to Right
Range: 0 to n-1, len(collection)

2.Negative Index Positions
Direction: Right to Left
Range: -1 to -n

Classifications of CDT:
Collection Data Type are classified into two types
1.ordered CDT
2.Unordered CDT or Random CDT

1.Ordered CDT
In ordered collection data type the data in memory will be stored in the same order as that we have defined in variable
Eg: String,Tuple,List,Dictionary

2.Unordered CDT:
In un-ordered CDT, in memory data will be stored in random order
Eg:Set

Indexing
It is a process of extracting individual elements from a given collection

Syntax for performing indexing:
 Variablename[Indexposition]

We can give both positive and negative index positions but internally negative indexes ae converted into positive indexes only

```
s='hai python'
s[0]
'h'
s[5]
'y'
s[8]
'o'
s[1]
'a'
s[3]
''
s[-2]
'o'
s[-5]
'y'
```

String

String is an collection of individual elements which are enclosed in a pair of quotation marks
String is a immutable data type
String is Ordered Collection Data Type
In strings indexing and slicing is possible
For representing single line strings we use single or double quotes
For representing multiple strings we use triple quotes

Syntax for declaring the single line string dt

Variable Name=‘element1,element2,element3,_ _ _ _ _ _element n’

or

Variable Name=“element1,element2,element3,_ _ _ _ _ _element n”

Syntax for declaring the multiple string data type

Variable Name=‘’’element1,element2,element3,_ _ _ _ _ _element n’’’

Slicing

It is Process of extracting multiple characters from a given collection

We can perform slicing in 2 ways:

1. **Positive slicing**
2. **Negative slicing**

Positive Slicing

It is the process of extracting multiple elements in the left to right direction

Syntax

Variable Name[startingindex:endingindex+1:updation]

Starting Index=0

Ending Index=len(collection)

Updation=+1

Starting Index position < Ending index position

Example: s[4:9:1]

4<9 ➔True

4+1=5<9 ➔True

5+1=6<9 ➔True

6+1=7<9 ➔True

7+1=8<9 ➔True

8+1=9<9 ➔False

Example: s=“Manikanta Machapalli”

```
s='manikanta machapalli'
s[0:9:1]
'manikanta’
s[0:11:3]
'min ‘
s[0:8:4]
'mk’

s[::-1]
'illapahcam atnakinam’

s[0:13:2]
'mnknamc'
s[10:5:1]
‘ '
s[10:14:4]
'm'
s[5:10:4]
'a '
s[7:12:1]
'ta ma'
```

Negative Slicing: It is the process of extracting multiple elements in the right to left direction

Variablename=[starting index:endingindex-1:updation]

In negative slicing default values are
Starting index ➔ -1
Ending Index ➔-(len(collection)-1)
Updation ➔ There is no default value

```
s='hai python'
s[-1:-5:1]
''
s[-1:-5:-1]
'noht'
s[-5:-10:-1]
'yp ia'
s[-7:-4:-1]
''
s[-2::-1]
'ohtyp iah'
s[::-1]
'nohtyp iah'
s[2:8:-1]
''
```

List: list is an collection of both homogenous and heterogeneous elements and each and every element is separated by , comma operator and enclosed in a pair of square brackets []
List is an mutable data type
List is an ordered collection data type
In list indexing and slicing is possible

```
L=[11,22,33,44,55]
L[2]
33
L[4]
55
L[0:3:]
[11,22,33]
L[-1:-4:-1]
[55,44,33]
L[::-2]
[55,33,11]
```

deleting the value space by index position

```
del variablename[indexposition]
del L[3]
L
[11,22,33,55]
```

Immutable data type: we cannot modified or we cannot changed
:Fixed length data type
Example: all single valued data types
string
tuple

Mutable Data type:
We can changed or we can modified
Variable length data type
Example: List
Set
Dictionary

- Tuple: Tuple is a collection of homogenous and heterogenous data in which each and every element is separated by ,[comma operator] and enclosed in a pair of ( )parenthesis
- Tuple is a Immutable Data type
- Tuple is an ordered collection data type
- Comma operator defines the tuple not the parenthesis
- Both indexing and slicing can be performed in tuple

- Syntax for defining multiple values in tuple

- Variable name=(element1,element2,element3,- - - - - - elementn)
- Variable name=element1,element2,_ _ _ _ _ _ _elementn
- Syntax for defining single value in tuple
- Variable name=(element1,) or
- Variable name=element1,

Example
T=(90,677,45,'hai','bye')
T[1]
677
T(2)
Error
T[-2]
'hai'

```
T[-3]
45
T[3][1]
‘a’
T[4][::-1]
‘eyb’
T[4][1::]
‘ye’
T[3][::2]
‘hi’
T[2]=100
Error
T[3][1]=‘l’
Error
```

Differences between tuple and list

TUPLE	List
tuple is enclosed in a pair of parenthesis (( ))	list is enclosed in a pair of square brackets ([ ])
tuple is an immutable data type	List is a mutable data types
tuple is a fixed length data type	List is a variable length data types
we can initialize a single in tuple (,) is mandatory. empty tuple we use tuple () (tuple function)	We can initialize a single value in list (,) is not mandatory For empty list we use square brackets

Set

1. Set is an collection of both homogeneous and heterogeneous data in which each and every element is separated by , comma operator and enclosed in a pair of { } (flower braces)

2. set is does not allows duplicate values
3. set is an unordered collection data type. So, we cannot perform indexing and slicing
4. For empty set we use set function
5. Set is mutable data type

6. Syntax for declaring set
7. Variable name={element1,element2,_ _ _ _ _element n}
8. Examples

```
S={'hai',99,23,10,23,11,34,'hai'}
S
{34,99,23,10,11,'hai'}
S
 {34,99,23,10,11,'hai'}
S={34,89,(34),78,23}
S
{89,34,78,23}
S={34,89,(34,),78,23}
S
{34,23,(34,),89,78}
```

Dictionary

Dictionary is a pair of keys and values declared inside a curly brackets({ }).

- Dictionary is a mutable data type.
- In the Dictionary data is in the form of pairs. So, we cannot perform indexing and slicing.
- Each and every keys and values are separated by colon (:).
- Each and every key value pairs are separated by comma (,).

Properties of keys

- We can store only immutable data types but we are advised to use string because string are defining as keys
- If we try to add duplicate values in dictionary, it will delete the old value and the new value will be assigned to that key

Properties of values

- We can store any type of data as values in dictionary
- We can give duplicate values as values in dictionary

```
d={'name':'mani','age':23,'state':'AP'}
d={'name':'mani','age':'mani','state':'AP'}
d
{'name': 'mani', 'age': 'mani', 'state': 'AP'}
d
{'name': 'mani', 'age': 'mani', 'state': 'AP'}
d={'name':'mani','name':23,'state':'AP'}
d
{'name': 23, 'state': 'AP'}
d={'name':'ashu','age':2}
d['name']
'ashu'
d['age']
2
d['name'][0]
'a'
d['name'][1::]
'shu'
d['name'][::3]
'au'
d['name'][::-1]
'uhsa'
```

```
d
{'name': 'ashu', 'age': 2}
d['age']=23
d
{'name': 'ashu', 'age': 23}
d
{'name': 'ashu', 'age': 23}
d['mobile']=9886282230
d
{'name': 'ashu', 'age': 23, 'mobile': 9886282230}
d.pop('name')
'ashu'
d
{'age': 23, 'mobile': 9886282230}
del d['mobile']
d
{'age': 23}
d={'name':'mani','age':'mani','state':'AP'}
d
{'name': 'mani', 'age': 'mani', 'state': 'AP'}
d.popitem()
('state', 'AP')
d
{'name': 'mani', 'age': 'mani'}
```

Type casting:
It is the process of converting one type of data into another type of data.

INTO WHICH	FROM WHICH	SYNTAX
Integer	float, Boolean, string containing only Integer	int (value/name)
float	Integer, Boolean, string containing int & float	float (value/ variable)
complex	Integer, Float, Boolean, string containing Int, float & complex	Complex (value/variable)
Boolean	All type of data false →0 & empty (space) CDT	bool(value/ variable)
String	All type of data	str (value/ Variablename)
List	string tuple, set dictionary (consider only keys)	List (value/ Variable)
Tuple	string list, set, dictionary (only keys)	Tuple (value/Variable)
Set	String List, Set, dictionary (only keys)	Set (value/Variable)
dictionary	List tuple set (in the form of pairs)	dict (value/ Variable)

```
int(20.0000000000)
20
int(True)
1
int(False)
0
int('8000')
8000
a='200'
type(a)
<class 'str'>
a=50.0000
int(a)
50
b=True
int(b)
1
c=False
int(c)
0
float(90)
90.0
float(True)
1.0
float(False)
0.0
float('20.000')
20.0
```

```
float('20')
20.0
float('mani’)
error
complex(12)
(12+0j)
complex(12,0)
(12+0j)
complex(12,4)
(12+4j)
complex(12,4,5)
TypeError
complex(90.00)
(90+0j)
complex(True)
(1+0j)
complex(False)
0j
complex('90')
(90+0j)
complex('90.00000000000')
(90+0j)
complex('90+90j')
(90+90j)
```

```
bool(0)
False
bool(9)
True
bool('mani'_)
SyntaxError: invalid syntax. Perhaps you forgot a comma?
bool('mani')
True
bool([])
False
bool(())
False
bool({})
False
bool(90)
True
bool(0)
False
bool(90.000)
True
bool(1+2j)
True
bool({'name':'mani'})
True
```

```
str(30)
'30'
str(30.0000)
'30.0'
str(1+2j)
'(1+2j)'
str(True)
'True'
str(False)
'False'
str([])
'[]'
str(())
'()'
str({})
'{}'
str({'name':'mani'})
"{'name': 'mani'}"
list('mani')
['m', 'a', 'n', 'i']
list((90,80,70,60,50,40,30,20,10))
[90, 80, 70, 60, 50, 40, 30, 20, 10]
list({10,20,30,40})
[40, 10, 20, 30]
```

```
list({'name':'mani'})
['name']
tuple('mani')
('m', 'a', 'n', 'i')
tuple([90,80])
(90, 80)
tuple({90,80})
(80, 90)
tuple({'name':'mani'})
('name',)
set('mani')
{'i', 'm', 'a', 'n'}
set([10,90])
{10, 90}
set({'name','mani'})
{'name', 'mani'}
set({'name':'mani'})
{'name’}
dict([('a',1)])
{'a': 1}
dict(((‘a',1),('b',2)))
{'a': 1, 'b': 2}
dict({('a',1)})
{'a': 1}
dict([{'a',1},{'b',2}])
{1: 'a', 2: 'b'}
```

Differences between function and method

Function	Method
Functions are the independent block of code which is used to perform one specific task	Method is also a function which is defined in a class
Function can be called or executed directly	In order to execute method we need either class or object
Syntax: functionname(value/variable)	Syntax: object reference.mn() Or Class reference.mn()
Def function() ---------- ----------	Class CN Def MN() ------- -------
Eg: type(vale/variable) Len,print,id,type	Eg: s.lower(),append,add,popitem

String built in methods:

Method name	syntax of method	Functionality
Lower()	s.lower()	it is used for converting each and every character of given string into lower case
Upper()	s.upper()	it is used for converting each and every character of given string into upper case
Title()	s.title()	First character of each and every character of given string will be converted into upper case and remaining will be in lower case
Capitilize()	s.captilize()	First character of entire string will be coverted into upper case remaining will be in lower case
Swapcase()	s.swapcase()	It is use for converting all lower case characters into upper case and all upper case characters into lower case

```
S=‘Hai PYTHON AND Django’
S.islower()
False
S.isupper()
False
S.istitle()
False
S.isalpha()
True
S.isdigit()
False
S.isalnum()
True
S=‘hai’
S.isalpha()
True
S.isdigit()
False
S=‘1234789’
S.isdigit()
True
S=‘9’
S.isdigit()
True
```

```
S=‘hai’
S1=‘988’
S2=‘hai786’
S.isalnum()
True
S1.isalnum()
True
S2.isalnum()
True
S3=‘haipython’
S3.isspace()
False
S4=‘ ‘
S4.isspace()
True
```

Method name	syntax of method	Functionality
islower()	s.islower()	Returns true if string satisfies lower method else returns false
isupper()	s.isupper()	Returns true if string satisfies upper method else returns false
istitle()	s.istitle()	Returns true if string satisfies title method else returns false
isalpha()	s.isalpha()	Returns true if string contains only alphabets else returns false
isdigit()	s.isdigit()	Returns true if string contains only digits else returns false

Method name	syntax of method	Functionality
isalnum()	s.isalnum()	Returns true if string contains only alphabets and digits or combination of both else returns false
isspace()	s. isspace()	Returns true if string contains only space else returns false
Strip() Rstrip() Lstrip()	s.strip() s.rstrip() s.lstrip()	These methods are used for removing the loading spaces from the given string
Split()	s.split([delimator],[count])	It is used for splitting the given string based on give delimator Delimator means space Count=how many time your delimator is repeated
replace()	s.replace(oldstring,newstring,[count])	It is used for replacing the existing characters with new characters

```
S=‘ hai python ‘
S.strip()
‘hai python’
S.rstrip()
‘ hai python’
S.lstrip()
‘hai python ‘
S=‘hai python’
S.split(‘t’)
[‘hai py’, ‘hon’]
S.split(‘a’)
[‘h’, ‘i python’]
S.split(‘h’)
[‘ ’,’ai pyt’, ‘on’]
S.split(‘h’,1)
[‘ ’,’ai python’]
S.split(‘h’,15)
[‘ ’,’ai pyt’, ‘on’]
S.split(‘w’)
[‘hai python’]
S=‘hai mani’
S.replace(‘h’,’b’)
‘bai mani’
S
‘hai mani’
s.replace(‘a’,’e’,1)
‘hei mani’
```

Method name	syntax of method	Functionality
Count()	s.count(substring,[startingindex],[endingindex])	It is used for returning how many times given substring is repeated in a given string
index()	s. index(substring,[SI],[EI]) **Left to right syntax** rindex()	It is used for returning the index position if value is present else if returns error as output
Find()	s.find(substring,[SI],[EI]) **Left to right syntax** rfind()	It is used for returning the index if value is present else it returns -1 as output
startswith()	s.startswith(Substring[SI],[Ei])	It is used for checking whether the string is starting with specified substring or not if it is starting it will return true else it returns false
endswith()	s.endswith(Substring[SI],[Ei])	It is used for checking whether the string is ending with specified substring or not if it is ending it will return true else it returns false

Method name	syntax of method	Functionality
format()	‘Content{},content{}’.format(value1,value2)	It is used for creating dynamic strings we need to create placeholders1 placeholders are created by {}
join()	‘gluecharacter’.join(iterable/collection)	It is used for creating new strings by joining the elements of given collection with glue characters

Built in methods in list

Method name	Syntax of method	Functionality
Count()	Count(value)	It is used for counting how many times a given element is repeated
Index()	Index(value,[SI],[EI])	It is used for finding the index position of given element
Clear()	Clear()	Clear method is used to deletes all the elements but not the memory address
append	Append(value) Value=>SVDT/CDT	It is used for adding a new element at the end of the list Svdt/CDT will be considered as only one element

Examples of the above built in methods

```
L=[11,22,33,11,78]
l.index(11)
0
l.index(11,2)
3
l.index(11,2,3)
Value error
l.count(11)
2
l.count(11,1)
Error
l.count(123)
0
```

Clear method examples

```
L=[11,999,33]
L.clear()
L
[]
```

Append examples

```
L=[23,89]
l.append(900)
L
[23,89,900]
L.append('hai')
L
[23,89,900,'hai']
L.append([90,78,67])
L
[23,89,900,'hai',[90,78,67]]
```

Shallow Copy	Deep copy
1. We can define the shallow copy by using slicing operator	1. We can define the deep copy by using importing copy method
2. In shallow copy both the objects are pointing to different memory address but the elements are pointing to same memory address	2. In deep copy both the objects are pointing to different memory address but the elements are pointing to different memory address.
3. If we try to modify one object other objects will not get affected but if we try to modify elements inside the object then the other objects will get affected 4. Ex:- l=[11,22,55,66,77] l1=l[::] print(l) print(l1) print(l is l1) print(l[0] is l1[0]) l1[0]=50 print(l) print(l1) Output [11, 22, 55, 66, 77] [11, 22, 55, 66, 77] False True [11, 22, 55, 66, 77] [50, 22, 55, 66, 77]	3. If we try to modify one object other objects will not get affected 4. Ex:- from copy import deepcopy l=[[1,2,3,4],['x','y','z','a’]] l1=deepcopy(l) print(l) print(l1) print(l is l1) print(l[0] is l1[0]) l1[0][1]=50 print(l) print(l1) Output [[1, 2, 3, 4], ['x', 'y', 'z', 'a’]] [[1, 2, 3, 4], ['x', 'y', 'z', 'a’]] False False [[1, 2, 3, 4], ['x', 'y', 'z', 'a’]] [[1, 50, 3, 4], ['x', 'y', 'z', 'a']]

Method Name	Syntax of method	Functionality
Extend	Extend(value) Value=>only CDT	First extracts individual Elements of given CDT & add them at the end of the list
Insert	Insert(Indexposition,value) Value=>SVDT/CDT	CDT/SVDT is considered as only one element It is used to add new elements at specified index position If specified IP>len(list) Then it will add the elements at the end of the list

```
L=[23,89]
L.extend(999)
Type error
L.extend('hai')
L
L.extend('hai')
L
[23,89,'h','a','i']
L.extend(['hello',78,21])
```

```
L
(23,89,'h','a','i','hello',78,21]
L.extend([100])
L
[23,89,'h','a','I','hello',78,21,100]
L=[11,22,33,44]
L.insert(2,100)
L
[11,22,100,33,44]
L.insert(1,'hai']
L
[11,'hai',22,100,33,44]
L.insert(4,[99,78,77])
L
[11,'hai',22,100,[99,78,77],33,44]
```

```
l
[10, 20, 30, 40, 50, 'mani', 'm', 'a', 'n', 'i']
l.extend(20)
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    l.extend(20)
TypeError: 'int' object is not iterable
l=['hai','mani','python',20,30,40]
l.extend('hai')
l
['hai', 'mani', 'python', 20, 30, 40, 'h', 'a', 'i']
l.extend('mani')
l
['hai', 'mani', 'python', 20, 30, 40, 'h', 'a', 'i', 'm', 'a', 'n', 'i']
l
['hai', 'mani', 'python', 20, 30, 40, 'h', 'a', 'i', 'm', 'a', 'n', 'i']
l.insert(6,50)
l
['hai', 'mani', 'python', 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i']
l.insert(0,1)
l
[1, 'hai', 'mani', 'python', 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i']
l
[1, 'hai', 'mani', 'python', 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i']
```

```
l
[1, 'mani', 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i', 11, 22]
l.remove('mani')
l
[1, 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i', 11, 22]
l.remove('h')
l
[1, 20, 30, 40, 50, 'a', 'i', 'm', 'a', 'n', 'i', 11, 22]
l.remove('a')
l
[1, 20, 30, 40, 50, 'i', 'm', 'a', 'n', 'i', 11, 22]
l.pop(5)
'i'
l
[1, 20, 30, 40, 50, 'm', 'a', 'n', 'i', 11, 22]
l.remove('n')
l
[1, 20, 30, 40, 50, 'm', 'a', 'i', 11, 22]
l.pop()
22
l.remove()
TypeError:
sort('mani’)
Type error
```

```
sort('mani')
NameError:
l=[90,1,2,3,4,60,40]
l.sort()
l
[1, 2, 3, 4, 40, 60, 90]
l=['mani','anusha','nagaraja','dileep','mounika']
l.sort()
l
['anusha', 'dileep', 'mani', 'mounika', 'nagaraja']
l.sort(reverse=True)
l
['nagaraja', 'mounika', 'mani', 'dileep', 'anusha']
sort('mani')
NameError: name 'sort' is not defined.
sorted('mani')
['a', 'i', 'm', 'n']
l.sorted()
typeerror
sorted((10,1,2,3))
[1, 2, 3, 10]
sorted('dileep')
['d', 'e', 'e', 'i', 'l', 'p']
sorted('dileep',reverse=True)
['p', 'l', 'i', 'e', 'e', 'd']
sorted('manitarak',reverse=True)
['t', 'r', 'n', 'm', 'k', 'i', 'a', 'a', 'a']
sorted('manitarak')
```

Pop	Remove
Pop method is used for removing the element based on index position	Remove method is used for removing the element based on value
Pop method will return the value	Remove method will not return the value
Pop method will take one argument	Remove method will take one mandatory argument
Syntax for pop:- pop(index position)	Syntax for remove:- remove(value)

```
l1
[11, 22, 33, 44, [90, 2, 3, 4]]
l.extend(l1)
l
[1, 'hai', 'mani', 'python', 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i', 11, 22, 33, 44, [90, 2, 3, 4]]
l1
[11, 22, 33, 44, [90, 2, 3, 4]]
l.pop(1)
'hai'
l.pop()
[90, 2, 3, 4]
l
[1, 'mani', 'python', 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i', 11, 22, 33, 44]
l.pop()
44
```

```
l
[1, 'mani', 'python', 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i', 11,
22, 33]
l.pop()
33
l.pop(2)
'python’
l
[1, 'mani', 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i', 11, 22]
l.remove('mani')
l
[1, 20, 30, 40, 50, 'h', 'a', 'i', 'm', 'a', 'n', 'i', 11, 22]
l.remove('h')
l
[1, 20, 30, 40, 50, 'a', 'i', 'm', 'a', 'n', 'i', 11, 22]
l.remove('a')
l
[1, 20, 30, 40, 50, 'i', 'm', 'a', 'n', 'i', 11, 22]
l.pop(5)
'i'
l
[1, 20, 30, 40, 50, 'm', 'a', 'n', 'i', 11, 22]
l.remove('n')
```

```
l
[1, 20, 30, 40, 50, 'm', 'a', 'i', 11, 22]
l.pop()
22
l.remove()
TypeError:
sort('mani’)
Type error
```

SORT

SORT	SORTED
Sort is a method of list	It is a normal built in function
Sort method is used only with list data type	Sorted can be used on any collection data type
Syntax for ascending order Objectreference.sort()	Syntax for ascending order Sorted(collection)
Syntax for descending order Objectreference.sort(reverse=true)	Syntax for descending order: Sorted(collection.reverse=True)
Output format is a list	Output format is a list

```
l=[1,90,3,50,60,4,2]
l.sort()
L
[1, 2, 3, 4, 50, 60, 90]
l.sort(reverse=True)
L
[90, 60, 50, 4, 3, 2, 1]
l.sorted()
Traceback error
sorted('mani’)
['a', 'i', 'm', 'n’]
sorted('siva’)
['a', 'i', 's', 'v']
```

```
sorted('siva',reverse=True)
['v', 's', 'i', 'a']
```

Built in methods in tuple

Count

1. Count(substring,[si],[ei])
2. It is used for returning how many times a given element is repeated in give tuple

Index

1. Index(substring,[si],[ei])
2. It is used for returning the index position if value is present else it returns as erro as an output

```
T=(90,78,67,56,90,34,99,90)
T.index(90)
0
T.index(90,2)
4
T.index(90,4)
4
T.index(90,5)
7
T.count(90)
3
T.count(99)
1
T.count(999)
0
```

Built in methods of set

1. Copy() --> it is used for performing the shallow copy
2. Clear() → it is used for clearing the elements of the given set

3. S={11,22,33,78,29,90,56}
4. S
5. {33, 22, 56, 90, 11, 29, 78}
6. S1=S.copy()
7. S
8. {33, 22, 56, 90, 11, 29, 78}
9. S1
10. {33, 22, 56, 90, 11, 29, 78}
11. id(S)
12. 2478716430016
13. id(S1)
14. 2478716873280
15. S1.clear()
16. S1
17. set()
18. S
19. {33, 22, 56, 90, 11, 29, 78}

Add()

It is used for adding the elements into set randomly
Syntax: add(value)

```
S={22,56,90,11,44,29,78}
S.add('hai')
S
{'hai',22,56,90,11,44,29,78}
S.add(999)
S
{11,78,22,90,29,'hai',999,44,56}
S.add([88,67])
Error list inside of tuple
S.add((44))
S
{11,78,22,90,29,'hai',999,44,56}
S.add((44,))
S
{11,78,22,90,29,(44,),'hai',999,44,56}
```

pop	remove	discard
Pop method will delete random element	Remove method is used for delete the element based on value	Discard method is used for delete the element based on value
pop method if we give index position it will throws a key error	If the specified element is present it will delete the value	If the specified element is present it will delete the value
Pop()	If the specified element is not present it will throws a key error	If the specified element is not present it will not perform any operation
	Syntax for remove:- remove(value)	Syntax for discard:- discard(value)

```
s={11,78,22,90,29,(40,),'hai',999,44,56}
s.pop()
999
s
{(40,), 11, 44, 78, 'hai', 22, 56, 90, 29}
s.pop()
(40,)
s.pop()
11
s
{44, 78, 'hai', 22, 56, 90, 29}
s.remove(29)
s
{44, 78, 'hai', 22, 56, 90}
s.remove('hai')
s
{44, 78, 22, 56, 90}
s.remove(90000)
Traceback (most recent call last):
  File "<pyshell#23>", line 1, in <module>
    s.remove(90000)
KeyError: 90000
s.discard(78)
s
{44, 22, 56, 90}
s.discard(90000)
```

union ()
syntax: baseset. union (set1,set2, --)
It is used for getting all the elements from the specified sets

intersection()
Syntax:baseset. intersection. (set1, set2,---)
It is used for getting the only common elements from the specified sets.

difference()
syntax:baseset. difference (set1,set 2,---)
It is used for getting the un-common elements from the baseset.

Note: By the above operations any of the specified s will not get modified.
Eg:
s = {11,22,33144,66}
s1= {22,100,200/300}
s2 = {33,44,1000}
s. union (s1)
{33,66,100,200,11,44,300,22}
S
{33,66,22,11,44}

S1
{200,100,2213003
s. union (s1,s2)
{33,66,100,200,1000,11,44,300,22}
s. intersection (s1)
{22}
s. intersection (s1,s2}
Set ()
s4= {22,99,77}
s. intersection (S1,S4)
{22}
s. difference(s1,s2)
{66,11}
s1.difference(s)
{200,100,300}

update()
Syntax: Baseset update (set1,set 2 ---)
It will perform Union operation and update the output into the Base set.

intersection-update()
Syntax: Baseset.intersection - update (set 1,set2,---)
It will perform intersection operation and update the output into the Baseset

difference - update()
Syntax: Base set difference update (set, set2,.----)
It will perform difference operation and update the output into the base set

symmetric-difference update()
Syntax: Base set.symmetric difference - update (set).
symmetric difference operation and update the out put into the base set.

Eg:
S= {11,22,33,44,66}
S1={100,200,300,33}
S2={22,44,1000}
S.update (S1)
{33,66,100,200,11,44,300,22}
S1
{200,33,100/300}
S.update (S1,S2)
S
{33,66,100, 200, 1000, 11,44,300,22}
S1. update (S)
S
{33,66,100,200,1000,11,300,44,22}
S
{33,66,100,200,1000,11,300,44,22}
S. intersection (s)
{33,66,100, 200, 1000, 11,300,44,22}
S={11/22,33,44,66}
S1 = {100,200,300,33}
S.intersection(S1)
{33}

```
S={11,22,33,44,66}
S1={100,200,300,33}
S.difference_update(S1)
S
{66,22,11,44}
S={11,22,33,44166}
>>>S1={100,200,300,33}
>>> S. difference-update (S1)
{66,22,11,44}
>>> S. difference - update (S1.S2)
{66,11}
>>> S= {11,22,33,44,66}
>>> S1={100,200, 300,33}
>>S2= {22,44,1000}
>>> S. symmetric-difference-update (S1)
S
{66,100,22,300,200,11,44}
S. symmetric-difference- update (S1,S2)
Type error, (only one set)
>>> S1. symmetric-difference-update(S2)
S1
{33,100,200,1000,300,44,22}
>>> S1 .symmetric - difference_update (S)
```

- SI
- {33,66,1000,11}

Issuperset():
It will check wheather the second set is a part of the first set or not

Issubset():
Is subset checks wheather the first set is a part of econd set or not.
If the first set is the part of second set then it will return a true or else it will return false.

Isdisjoint():
It checks wheather two sets contains any common elements it returns false or else it returns true

```
s={11,22,33,44}
s1={22,44}
s2={22,11,66}
s.issuperset(s1)
True
s2.issuperset(s1)
False
s1.issuperset(s)
False
s1.issuperset(s2)
False
s.issubset(s1)
False
s1.issubset(s)
True
s.isdisjoint(s1)
False
s2.isdisjoint(s1)
False
s3={100,200,300}
s3.isdisjoint(s1)
True
s2.isdisjoint(s3)Trues1.isdisjoint(s3)True
```

Built in methods in dictionary
Key()
keySyntax: key()
It is used for getting the keys of specified dictionary
Value()
Syntax:value()
It is used for getting the value of specified dictionary

Items()
It is used for getting the both keys and values of specified dictionary

Eg: d={‘name’:’ashu’,’age’:2}
d.key()
dict_keys([‘name’,’age’])
d.values()
dict_values([‘’ashu’,2])
d.items()
dict_items([(‘name’,’ashu’),(‘age’,2)])

Copy()
It is used for performing shallow copy

Clear(): it will delete only the elements from the given dictionary

```
d={‘name’:’ashu’,’age’:2}
d1=d.copy()
d1
{'name': 'ashu', 'age': 2}
d
{'name': 'ashu', 'age': 2}
d.clear()
d
{}
d1
{'name': 'ashu', 'age': 2}
d1.clear()
d1
{}
```

Get()	Setdefault()
If the specified key is present it will display the value	If the specified key is present it will display the value
2. If the specified key is not present it will take the default value	2. If the specified key is not present it will update the dictionary key and values
3. Syntax :- get(keyname,[default value])	3. Syntax :-set default(keyname,[default value])
4. If the default value is not present it will give None as an output	4. If the default value is not present it will update the actual dictionary
Ex:- d={'name':'hari','age':5} d {'name': 'hari', 'age': 5} d.get('name’) 'hari’ d.get(‘age’) 5 d.get(‘AGE’,15) 15	Ex:- d={'name':'mahesh','address':'hyderab ad','state':'telangana’} d {'name': 'mahesh', 'address': 'hyderabad', 'state': 'telangana’} d.setdefault('name’) 'mahesh’ d.setdefault('age',55) 55 d {'name': 'mahesh', 'address': 'hyderabad', 'state': 'telangana', 'age': 55}

Update method:
It is used for performing the actual dictionary with multiple key and value pairs at a time
Note: It is used for merging dictionaries
Syntax:
Baseddictionary.update({‘key1’:’value1’,’key2’:’value2’})

If the specified key is present then it will update the value
If the specified key is not present then it will create new key value pair

```
d
{'name': 'ashu', 'age': 2, 'namee': 'nikky', 'NAME': 'MANI', 'ages': 80}
d.update({'mobile':9886282230,'gender':'male’})
d{'name': 'ashu', 'age': 2, 'namee': 'nikky', 'NAME': 'MANI', 'ages': 80, 'mobile': 9886282230, 'gender': 'male’}
d.update({'mobile':7337461778})
d{'name': 'ashu', 'age': 2, 'namee': 'nikky', 'NAME': 'MANI', 'ages': 80, 'mobile': 7337461778, 'gender': 'male’}
d.update({'name':'praveen’})
d{'name': 'praveen', 'age': 2, 'namee': 'nikky', 'NAME': 'MANI', 'ages': 80, 'mobile': 7337461778, 'gender': 'male’}
d.pop('namee’)
'nikky’
d
{'name': 'praveen', 'age': 2, 'NAME': 'MANI', 'ages': 80, 'mobile': 7337461778, 'gender': 'male'}
```

Pop()	Popitem()
Syntax:pop(keyname)	Syntax:popitem()
It is used for deleting the key value pairs based on specified key	It is used for deleting the last key and value pair by default
If key is present then it will delete else it will throw me an error	If dictionary is empty it throws an error
	Output format of popitem is a tuple

```
d={'name': 'praveen', 'age': 2, 'namee': 'nikky', 'NAME': 'MANI', 'ages': 80, 'mobile': 7337461778, 'gender': 'male’}
d.pop('namee’)
'nikky’
d
{'name': 'praveen', 'age': 2, 'NAME': 'MANI', 'ages': 80, 'mobile': 7337461778, 'gender': 'male’}
d.pop('NAME’)
'MANI’
d
{'name': 'praveen', 'age': 2, 'ages': 80, 'mobile': 7337461778, 'gender': 'male’}
d.pop('NAME’)
Traceback KeyError
d.popitem()
('gender', 'male’)
d
{'name': 'praveen', 'age': 2, 'ages': 80, 'mobile': 7337461778}
```

Operators:

These are used for performing operations on operands

1.Arithmetic operators

2.Logical operators(and,or,not)

3.Relational operators(>,<,>=,<=,==,!=)

4.Bitwise operators(&,|,^,~,>>,<<)

5.Assignment operators(+=,-=,*=,/=,%=,**=,//=)

6.Membership operators(in operator, not in operator)

7.Identity Operators(is,isnot)

1.ARITHMETIC OPERATORS:

Addition(+)→addition operators are used to adding the values

Substraction(-)→Substraction operators are used to substract the values

Multiplication(*)→Multiplication operator are used to multiply the values

Power Operator(**)→a**b→a to the power of b→3**3→27

Normal Division(/)→performs the division and gives quotient as output in the form of float

Floor Division(//)→performs the division and eliminates the decimal values from the output

Modulur division(%): it is used for returning the remainder

```
a=90
b=10
a+b
100
10+20
30
50+900
950
a=90
b=80
a+b
170
a-b
10
a*b
7200
a=10
b=2
a/b
5.0
a//b
5
10/2
5.0
100/5
20.0
100//5
20
```

```
10/2
5.0
10//2
5
2**2
4
3**3
27
4**2
16
2**2**2
16
2**3**2
512
2**2**4
65536
2**1**2**3**2
2
2
2
2**3**2
512
2**2**2
16
2**2**2**2
65536
```

Relational Operators

1. Relational operators are used for returning a Boolean value as an output
2. These relational operators are used in conditional statements
3. >,>=,<,<=,!=
4. Number to number comparision we can do it easily but when we compare strings then comparision is done based on the ASCII Values of the characters in case of CDT to CDT it will compare first element.

Example:1>2
False
1<2
True
100>12
True
100<90
False
'Mani'>'Nani'#ASCII VAlues A=65,Z=90,a=97,z=122
False
ord('A')
65
ord('Z')
90
ord('a')
97
ord('b')
98
ord('z')
122
[10,20,33]>[20,33,44]
False
[20,10,33]>[90,10,20]
False
'mani'>'nani'
False

'nani'>'mani'
True

Logical Operators

Logical operators are used for returning a Boolean value as an output

These logical operators are used in conditional statements

AND OPERATOR: if both conditions is true then it will print true or else it will prints false

Operand1	Operator	Operand2	Result
True	AND	True	True
True	AND	False	False
False	AND	True	False
False	AND	False	False

1 and 4
4
‘hai’ and []
[]
‘ ‘ and 90
‘ ‘
[] and ‘ ‘
[]

OR OPERATOR: If one condition is true then it will print true condition, i.e result will be true

operand1	operator	operand2	result
True	OR	True	True
True	OR	False	True
False	OR	True	True
False	**OR**	**False**	**False**

1 or 5
1
6 or 0
6
‘ ‘ or 78
78
[] or ‘‘
‘’

Not: not is used for performing negation operation
Syntax: Not operand 1
Eg:
Not 7
False
Not []
True
Not ‘hello’
False
Not [‘‘]
False

Assignment Operators:

These are used for assigning some values to the variable by using = operator we can assign the value for a variable

A+=B

A-=B

A*=B

A**=B

A/=B

A//=B

A%=B

Eg: A=10

A+=90

A

100

A-=50

A

50

A*=2

A

100

A/=3

A

33.333

A//=2

A

16

A**=2

A

256.0

Membership operator
It is used for checking wheather the element is part of given collection or not

IN OPERATOR
It returns True if value is present in given collection else returns False
Syntax: SVDT/CDT in collection

NOT IN OPERATOR:
It returns true if value is not present in givem collection else returns false
Syntax:
SVDT/CDT not in collection

Eg: 1 in 123
type error
1 in ‘123’
Type error
‘1’ in ‘123’
True
[1] in [2,3,1,4]
False
[1] in [2,3,[1],4]
True
‘hai’ in [11,22,33,44]
False

123 in (1,2,3,4)
False
2 in {‘name’:’ashu’,’age’:2]
False
2 not in {‘name’:’ashu’,’age’:2}
True
[1] not in [2,3,[1],4]
False

Identity operators:
If multiple values are having same below mentioned then they will point to same memory address
1.Integers till 256
2.Strings
== operator compares values directly is,is not operators compares memory address
These operators are used for comparing the memory addresses

IS operatos: it returns True if variables are pointing to same memory address else returns False
Syntax: Operand1 is Operand2

IS NOT: it returns true if variables are not pointing to same memory address else returns False
Syntax: Operand1 is not operand2
A=256
B=256
A==b

```
True
A is B
True
C=257
D=257
C==D
True
C IS D
False
S=‘hai’
S1=‘hai’
S is S1
True
L=[11,22,33]
L1=[11,22,33]
L is not L1
True
T=(11,22,33)
T1=(11,22,33)
T is T1
False
```

Bitwise Operators:

These operators are used for performing operations on the binary values of given number

Bitwise &→ Returns 1 when both operands are 1

Eg: 1&2

0

2&3

2

```
1
0 0 1 0
0 0 1 1
------------
0  0  1 0→ 2
-------------
```

```
2&3
0  0  0  1
0  0  1  0
----------------
 0 0  0   1→0
----------------
```

Bitwise /→return 0 when both operands are 0

1|2

3

2|3

3

```
0  0  1  0
0  0  1  1
-----------------------
0  0  1   1→3
-----------------------
```

```
0  0  1  0
 0  0  1  0
-----------------------
 0   0  1   1→3
-------------------------
```

Bitwise ^: Return 0 when both operands are same

Eg: 1^2

3

2|3

1